



UNICAMP

Universidade Estadual de Campinas

Instituto de Matemática, Estatística e
Computação Científica

Método Primal Simplex

Prof. Kelly Poldi

Luís Roberto Marques Júnior RA:173216

Introdução

O projeto computacional proposto tem por objetivo a implementação de um algoritmo - nesse caso executado em Python - que resolva um problema de programação linear (PPL) na forma padrão, dada por:

$$\begin{array}{ll} \text{minimização} & f(\mathbf{x}): \mathbf{c}^t \mathbf{x} \\ \text{sujeito a:} & \mathbf{Ax} = \mathbf{b} \\ & \mathbf{x} \geq 0, \end{array}$$

que tem as seguintes entradas:

- quantidade de linhas (restrições);
- quantidade de colunas (variáveis);
- vetor de custos
- vetor dos recursos
- matriz com os coeficientes das restrições

é esperado na saída que o programa consiga resolver tal que:

- se o problema for ilimitado, que informe.
- se tiver solução, imprimir vetor $\mathbf{x}^*$ com os valores ótimos, e o valor ótimo de $f(\mathbf{x}^*)$.

Método Primal Simplex

Como já foi mencionado e disponibilizado o algoritmo do Primal Simplex em aula (**ref. aula 10**), devemos seguir alguns passos bem específicos.

Primeiramente, devemos escrever a matriz $\mathbf{A}$ que é composta pelos coeficientes do problema proposto, juntamente com o vetor transposto de $\mathbf{b}$ (recursos) e $\mathbf{c}$ (custos). Em seguida, partimos de uma partição básica que é supostamente dada - o algoritmo aqui em questão não faz nenhum apelo à Fase I do método -, juntamente com sua partição não básica.

A partir daqui, seguiremos, rigidamente, 7 passos, que são eles:

1. Calcular o sistema linear proposto pela matriz básica, vetor de recursos e variáveis básicas $\mathbf{B}^* \mathbf{x}_b = \mathbf{b}$;
2. Calcular o vetor multiplicador do simplex, conhecido como lambda, através da solução de $\mathbf{B}'^* \lambda = \mathbf{c}_b$.
3. Calcular os custos reduzidos das variáveis-não básicas expresso por $\mathbf{c}_n - \lambda' * \mathbf{a}_n$; com esse valor, decidir qual variável $\mathbf{a}_n$ entra na base.

4. Calcular a direção simplex por $\mathbf{B}^* \mathbf{d} = \mathbf{a}_n$.
5. Calcular o tamanho do passo dado por $x b_i / d_i$, assim decidindo quando variável sai da base.
6. atualizar a partição básica ($\mathbf{B}$), e a não-básica.
7. Recomeça o ciclo a partir de 1.

Esse algoritmo se estende até que uma de 3 condições de parada seja observada, e são elas:

- a. todos os custos relativos das variáveis não-básicas são positivos maiores que zero, portanto essa é a solução ótima.
- b. no cálculo do passo, só temos valores $x b_i / d_i = 0$, portanto, a solução é ilimitada.
- c. nota-se uma repetição das mesmas trocas de bases ao longo do processo; nesse caso, uma perturbação no vetor de recursos ($\mathbf{b}$), é necessária.

Algoritmo

O código foi dividido em duas partes principais: a definição da função **primal_simplex** e a inserção de dados que será feita pelo usuário.

A definição da função `primal_simplex` é feita com três argumentos no formato matricial; essa função contém a implementação do algoritmo Simplex. Com os argumentos dados ela retorna a solução ótima -se houver - ou uma mensagem indicando que o problema é ilimitado.

O código espera a inserção de quatro argumentos: o número de linhas e colunas, o vetor de recursos, o vetor de custos e os coeficientes da matriz A que será trabalhada.

Passo a passo do algoritmo, temos:

Inicialização da Base: ele começa inicializando uma base factível, que consiste nas variáveis de folga associadas às restrições. Isso é feito por meio da matriz `base_factivel`.

Loop Principal: O código entra em um loop que executa repetidamente os seguintes passos:

- a. Cálculo dos custos reduzidos: calcula os custos reduzidos para todas as variáveis, o que indica quanto a função objetivo pode melhorar se a variável for incluída na solução.

b. Verificação de otimalidade: verifica se os custos reduzidos de todas as variáveis são não negativos. Se forem, isso indica que uma solução ótima foi encontrada, e o algoritmo termina.

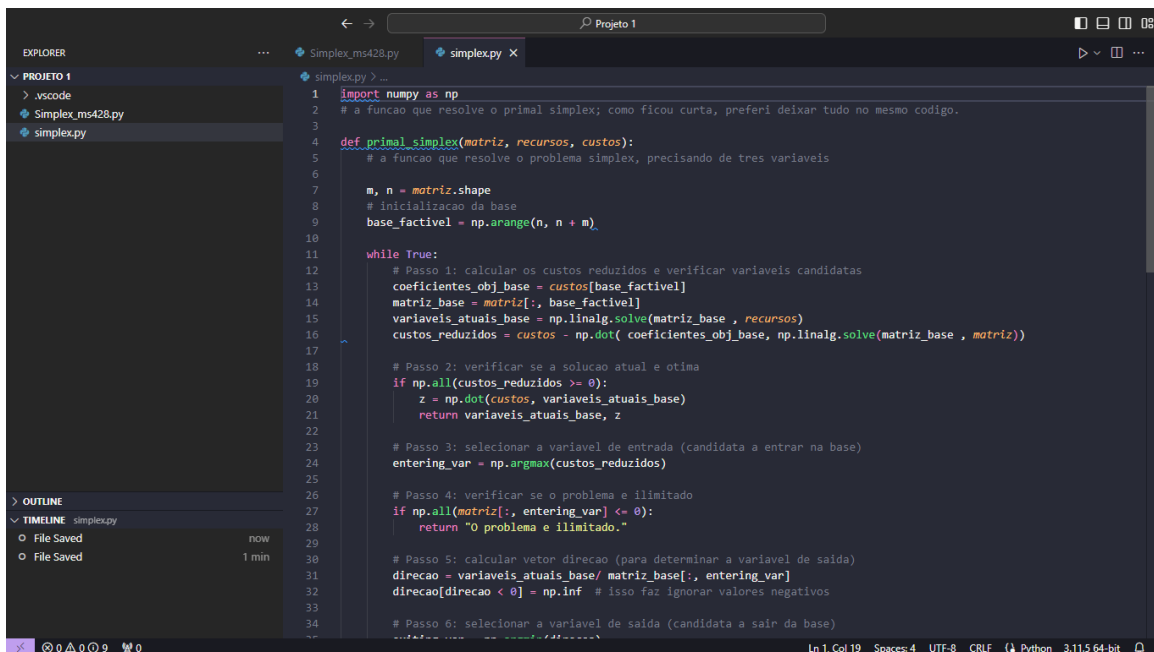
c. Seleção da variável de entrada: Se os custos reduzidos não forem todos não negativos, o algoritmo seleciona a variável de entrada (a que tem o maior custo reduzido negativo) para entrar na base.

d. Verificação de problema ilimitado: Verifica se o problema é ilimitado, ou seja, se não existe uma solução ótima finita. Isso é feito verificando se todos os coeficientes correspondentes à variável de entrada são não positivos.

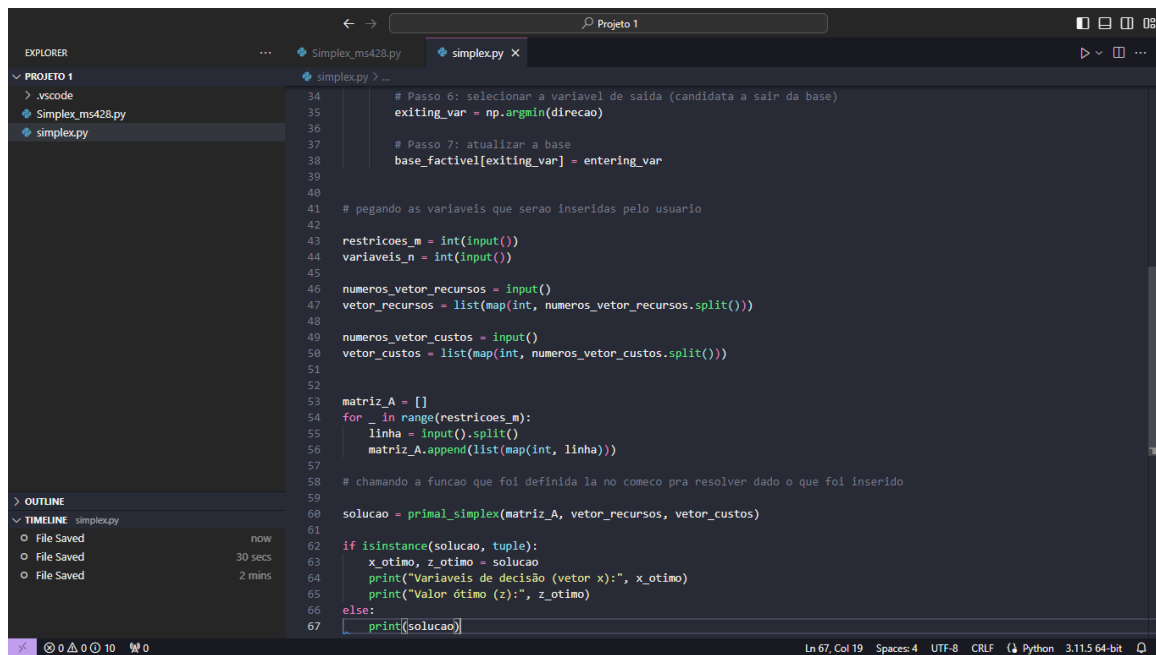
e. Cálculo do vetor direção: Calcula o vetor direção para determinar a variável de saída (a que sai da base). Isso é feito dividindo as variáveis atuais pela coluna correspondente da variável de entrada e ignorando valores negativos.

f. Seleção da variável de saída: Seleciona a variável de saída (a que sai da base) com base no vetor direção calculado.

g. Atualização da base: Atualiza a base factível com as variáveis selecionadas. Fim do Algoritmo: O algoritmo termina quando encontra uma solução ótima ou determina que o problema é ilimitado.



```
1 import numpy as np
2 # a funcao que resolve o primal simplex, como ficou curta, preferi deixar tudo no mesmo codigo.
3
4 def primal_simplex(matriz, recursos, custos):
5     # a funcao que resolve o problema simplex, precisando de tres variaveis
6
7     m, n = matriz.shape
8     # inicializacao da base
9     base_factivel = np.arange(n, n + m)
10
11     while True:
12         # Passo 1: calcular os custos reduzidos e verificar variaveis candidatas
13         coeficientes_obj_base = custos[base_factivel]
14         matriz_base = matriz[:, base_factivel]
15         variaveis_atuais_base = np.linalg.solve(matriz_base, recursos)
16         custos_reduzidos = custos - np.dot(coeficientes_obj_base, np.linalg.solve(matriz_base, matriz))
17
18         # Passo 2: verificar se a solucao atual e otima
19         if np.all(custos_reduzidos >= 0):
20             z = np.dot(custos, variaveis_atuais_base)
21             return variaveis_atuais_base, z
22
23         # Passo 3: selecionar a variavel de entrada (candidata a entrar na base)
24         entering_var = np.argmax(custos_reduzidos)
25
26         # Passo 4: verificar se o problema e ilimitado
27         if np.all(matriz[:, entering_var] <= 0):
28             return "O problema e ilimitado."
29
30         # Passo 5: calcular vetor direcao (para determinar a variavel de saida)
31         direcao = variaveis_atuais_base / matriz_base[:, entering_var]
32         direcao[direcao < 0] = np.Inf # isso faz ignorar valores negativos
33
34         # Passo 6: selecionar a variavel de saida (candidata a sair da base)
```



```
34 # Passo 6: selecionar a variavel de saida (candidata a sair da base)
35 exiting_var = np.argmax(direcao)
36
37 # Passo 7: atualizar a base
38 base_factive[exiting_var] = entering_var
39
40
41 # pegando as variaveis que serao inseridas pelo usuario
42
43 restricoes_m = int(input())
44 variaveis_n = int(input())
45
46 numeros_vetor_recursos = input()
47 vetor_recursos = list(map(int, numeros_vetor_recursos.split()))
48
49 numeros_vetor_custos = input()
50 vetor_custos = list(map(int, numeros_vetor_custos.split()))
51
52
53 matriz_A = []
54 for _ in range(restricoes_m):
55     linha = input().split()
56     matriz_A.append(list(map(int, linha)))
57
58 # chamando a funcao que foi definida la no comeco pra resolver dado o que foi inserido
59
60 solucao = primal_simplex(matriz_A, vetor_recursos, vetor_custos)
61
62 if isinstance(solucao, tuple):
63     x_otimo, z_otimo = solucao
64     print("Variaveis de decisao (vetor x):", x_otimo)
65     print("Valor otimo (z):", z_otimo)
66 else:
67     print(solucao)
```

Por fim, o código foi anexado junto com o relatório no classroom, em pdf.